

Технологии программирования



Омниссия

Единой верой

Простой оверрайд

- Одинаковая сигнатура функции в классах родителях и наследниках.
- При использовании объекта наследника будет вызвана функция того типа из которого она вызвана. Это раннее или статическое связывание.

```
void TellMeYourName() /*class Human*/ {  
    printf("nameless human\n");  
};
```

```
void TellMeYourName() /*class Ultramarine*/ {  
    printf("Marneus Calgar\n");  
};
```

Отличие от оверлоада

- Сигнатура оверрайдов полностью идентична.
- Оверрайд можно создать только для функций в классах связанных наследованием.

```
Human human;  
Ultramarine ultramarine("Marneus Calgar");  
  
ultramarine.TellMeYourName(); // "Marneus Calgar"  
human.TellMeYourName();      // "nameless human"  
  
Human* adept = &ultramarine; // upcast - это всё ещё Марнеус  
adept->TellMeYourName();      // "nameless human"
```



Принцип полиморфизма нарушен. В контексте родителя мы не можем работать, нам нужно знать тип объекта.

Аугментация

Полиморфизм

Возможность использовать классы потомки в контексте, который был предназначен для класса – предка.

Взаимодействие с объектами классов ведется только на уровне их интерфейсов, игнорируя внутреннее устройство. Нам достаточно интерфейса и нам всё равно что за объект за ним скрывается, важно только то, что мы можем вызвать любую функцию интерфейса и она будет выполнена в соответствии с нашими ожиданиями (теоретически...)

Аугментация в действии

Полиморфизм в общих словах

Создадим абстракцию на основании общих признаков:

Человек

Космодесантник

Ультрамарин

Человек может ходить, прыгать, стрелять, выполнять действия.

Рассмотрим действия Космодесантника и Ультрамарина, в общем смысле они могут совершать те же действия, что и человек, различие только в реализации этих действий.

Отдавая приказ (вызывая функцию) идти, нам не важно, обратились мы к нему как к человеку или как к Ультрамарину, важно то, что он выполнит эту задачу, и сделает это как Ультрамарин.



Истинная сила

Виртуальная функция: `virtual`

- Определяет виртуальные (полиморфные) функции в классе.
- Определяется типом создания объекта, а не типом из которого она вызвана. Это позднее или динамическое связывание.
- Имеет полный доступ к полям и методам своего класса, вне зависимости от того с какого уровня иерархии их вызвали.
- Может быть любой - `public`, `protected`, `private` - хотя последнее при правильной инкапсуляции не имеет смысла.
- Виртуальной функцию можно объявить на любом уровне наследования, расширяя интерфейс классов наследников:
 - Базовым классом для виртуальной функции будет тот, в котором функция объявлена в первые.
 - Из класса уровнем выше, эту функцию вызвать нельзя, она там не существует.

```
class Human
{
public:
    virtual void Walk(uint32_t steps); {printf("Human walks\n");};
    // виртуальная функция. в классах наследниках ее реализация может отличаться
};
```



Апгрейды

Переопределение виртуальных функции. Override.

- В классе наследнике виртуальная функция может быть переопределена - override.
- Если override в классе наследнике нет, будет вызываться первая функция определенная вверх по иерархии (к родителям).
- Не обязательно переопределять виртуальную функцию во всех наследниках.
- Требование к override функции - полное совпадение сигнатуры. Override можно не маркировать, или использовать ключевые слова virtual и override.

```
void Walk(uint32_t steps);           // override для родительской функции Walk
virtual void Walk(uint32_t steps); // то же самое
void Walk(uint32_t steps) override // override для Walk с указанием компилятору проверить сигнатуру
```

- Ключевое слово override - директива компилятору проверить, что сигнатура соответствует сигнатуре виртуальной функции любого родителя в цепочке наследования. Предпочтительно использовать override.

```
class Ultramarine : public SpaceMarine
{
public:
    void Walk(uint32_t steps) override {printf("Ultramarine marches %d\n", steps);};
}
```



Ограничение модификаций

Final для функций

- Аналогично final для классов, прекращает цепочку наследования.
- Виртуальная функция помеченная final не может быть переопределена в классах наследниках.
- Все классы наследники будут использовать последнюю - финальную реализацию.
- Ключевое слово final - директива компилятору: пресечь последующее переопределение функции.

```
void PraiseTheEmperor() override final {printf("For the Emperor!!! for the living God!!!\n");};
```



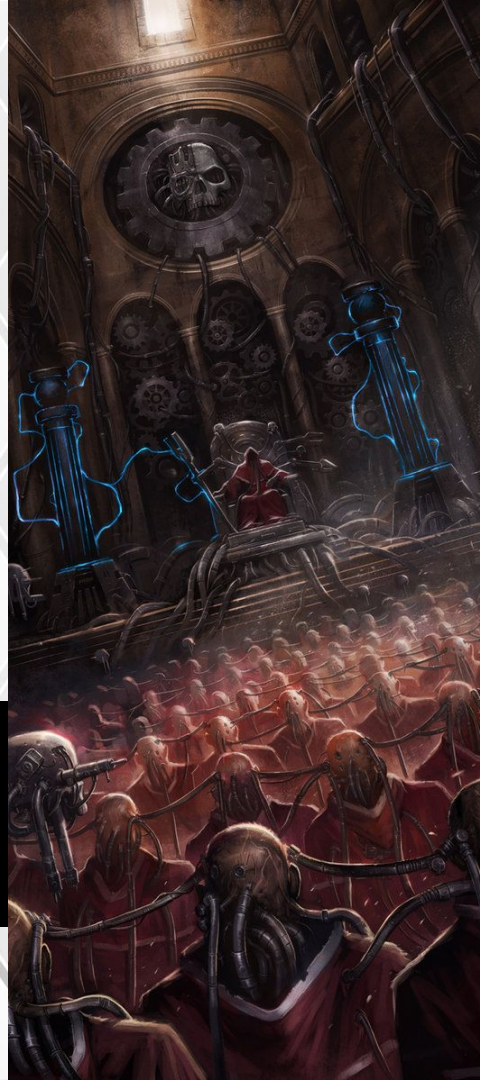
Обращаясь к истокам

Вызов функций родительских классов

- В классе наследнике можно вызвать функцию родителя.
- Вызвать функцию родителя можно с любого уровня иерархии.
- Вызвать можно любую функцию родителя (кроме приватных).
- Функция не обязательно должна быть виртуальной.
- Можно вызвать не виртуальную функцию с оверрайдом, в данном случае будет вызвана функция класса к которому указано обращение.
- Если функция к которой обратились отсутствует в указанном классе, будет вызвана ближайшая по иерархии вверх.

```
void Walk(uint32_t steps) override // class Ultramarine
{
    printf("Ultramarine Marches %d\n", steps);
    Human::Walk(steps); // вызов функции родительского класса SpaceMarine
};
```

Злоупотребление обращением к родительским функциям может привести к наследованию ради code reuse, что является плохой практикой.



Вопрос

Итак, что получится

```
Human justMan;  
Ultramarine ultramarine("Marneus Calgar");  
  
justMan.Walk(10);           // "Human Walks"  
ultramarine.Walk(10);       // "Ultramarine Marches"  
  
Human* human = &ultramarine; // обратимся к ультрамарину как к человеку  
human->Walk(10);             // "Ultramarine Marches"
```

Важно! При вызове виртуальной функции учитывается то, какого типа объект был создан, а не к какому типу приведен в момент вызова функции.

Призвать лучших

Ковариантность

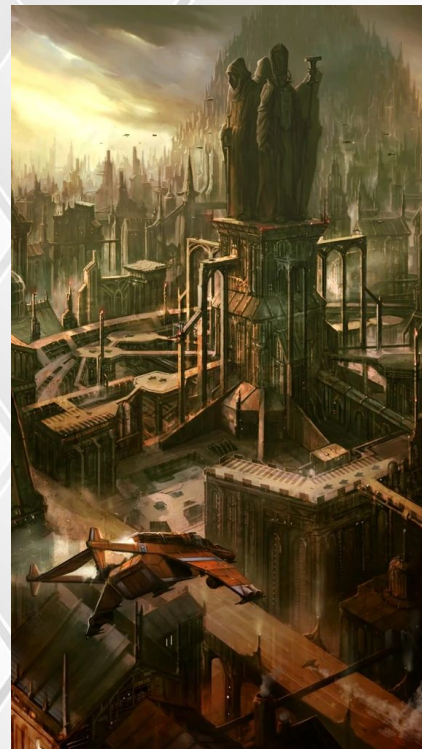
- Возможность в классе наследнике возвращать из виртуальной функции тип, унаследованный от типа возвращаемой функции родительского класса.
- Оверрайд с идентичными параметрами, но (формально) различными retval.

```
class Order {  
public:  
    virtual SpaceMarine* RequestMarine();  
};  
  
class OrderUltramarines : public Order {  
Public:  
    Ultramarine* RequestMarine() override; // Ultramarine - наследник SpaceMarine  
};
```

- В случае приведения к родительскому типу возвращаемое значение функции оверрайда будет автоматически приведено к SpaceMarine.
- Использовать сигнатуру класса наследника нельзя.

```
Order *theUltramarines = new OrderUltramarines;  
Ultramarine* adeptus = theUltramarines->RequestMarine();  
//Еггг, будет попытка downcast, чего без спец операторов сделать нельзя
```

```
inline virtual SpaceMarine *Order::RequestMarine()
```



В возвращаемых значениях менять порядок наследования нельзя, только вниз.

Из варпа



Деструктор в наследовании

- Как и любая другая функция, деструктор подчиняется правилам раннего и позднего связывания.

```
Ultramarine* ultramarine = new Ultramarine("Marneus Calgar");  
SpaceMarine* justMarine = ultramarine;  
delete justMarine;
```

- В случае не виртуального деструктора объект удалится как объект класса SpaceMarine, т.е. вызовется цепочка деструкторов начиная от SpaceMarine вверх.
- Если для конструирования класса Ultramarine выделялась память, она останется неосвобожденной. **Memory leak.**
- Чтобы избежать memory leak, достаточно связать деструкторы классов в цепочке наследования поздним связыванием. Или объявить деструктор самого верхнего родителя виртуальным

```
virtual ~Human() {};
```

Простое правило, при наличии хоть одной виртуальной функции в классе конструктор надо объявить виртуальным. Некоторые анализаторы кода не виртуальный деструктор в классе с виртуальными функциями считают ошибкой.

Рассы

Чисто виртуальная функция.

- Чисто виртуальные функции предназначены для определения интерфейса - общих характеристик классов в цепочке наследования.

```
virtual void DoSomething(uint32_t steps) = 0;
```

- Чисто виртуальные функции можно смешивать в определении класса с обычными и виртуальными функциями.
- Создать объект класса с чисто виртуальной функцией нельзя.
- Вызвать чисто виртуальную функцию напрямую нельзя. Косвенный вызов приведет к "Pure virtual function call" ошибке.
- Интерфейсы - состоят только из чисто виртуальных методов.

```
class Humanoid {                // для определения интерфейсов лучше использовать структуры.  
public:                          // слово public тогда не нужно. Интерфейс получается чище.  
    virtual void Walk(uint32_t steps) = 0;  
    virtual void Jump() = 0;  
    virtual void Sit() = 0;  
    virtual ~Humanoid() {};  
}
```

Несмотря на то, что в классах можно смешивать чисто виртуальные и обычные методы, лучше этого избегать. Придерживаться парадигмы интерфейсов и их наследников.



Все как один

```
std::vector<Human*> squadron; // Смешанный отряд

squadron.push_back(new SpaceMarine("Fabian", 1));
squadron.push_back(new Ultramarine("Marneus Calgar"));
squadron.push_back(new Terminator("Rogo Victurix"));

for (auto& marine : squadron)
{
    marine->Walk(10);    // Не важно кто они, но все пойдут, и сделают это каждый по своему
}
```

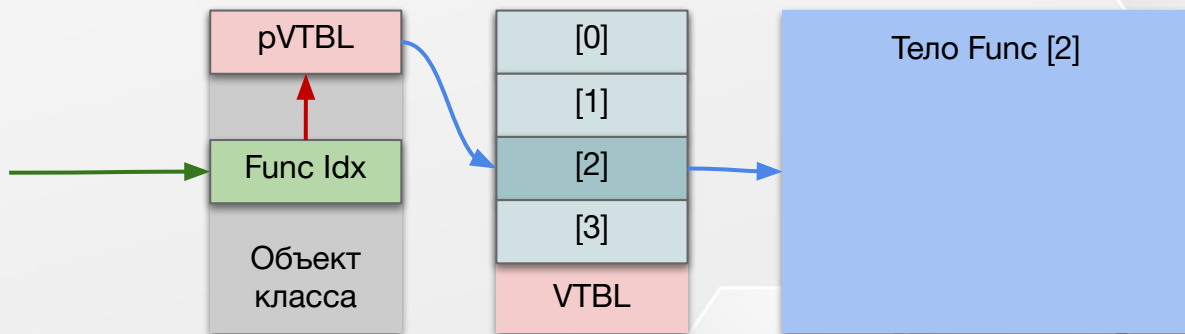


WARP



VTBL Virtual table - таблица виртуальных функций

- Класс в объявлении которого присутствуют виртуальные функции формирует таблицу виртуальных функций.
- VTBL присутствует в единственном экземпляре для каждого типа класса.
- В таблице содержатся указатели на виртуальные функции этого класса.
- Все объекты полиморфных классов содержат указатель на таблицу виртуальных функций.
- Указатель на таблицу - первое поле в объекте класса.



Указатель на VTBL и является источником полиморфизма. Он не зависит от типа к которому приведен объект класса наследника, указатель всегда указывает на VTBL класса, объект которого был создан.

Орден меня призывает
Увидимся через 2 недели.

